

DOCUMENTO DE DISEÑO

Sistema de Registro y Recomendación de Equipo para Wing Foil

Programación para Ciencias del Comportamiento — Junio 2026

1. OBJETIVO DEL PROGRAMA

El programa permite a deportistas de wing foil registrar sus sesiones de navegación y recibir recomendaciones personalizadas de equipo basadas en su propio historial. Aprende de las sesiones pasadas del usuario y sugiere qué wing usar según las condiciones del día (viento, dirección y ubicación), buscando las sesiones más similares en el historial y recomendando el wing con mayor sensación promedio en esas condiciones.

2. INPUTS

2.1. Dataset histórico de sesiones

Archivo Excel (dataset_sesiones.xlsx) en la carpeta data/. Cada fila representa una sesión registrada. Las columnas son:

Campo	Tipo	Unidad	Descripcion
Fecha	str	AAAA-MM-DD	Fecha de la sesión
Duración (min)	float	minutos	Duración de la sesión (> 0)
Ubicación	str	—	Nombre del spot de navegación
Vel.viento	float	nudos	Velocidad del viento (> 0)
Dir viento	str	—	Dirección del viento (Norte, Sur, Este, Oeste, Noreste, Sudeste, Noroeste, Sudoeste)
Wing	str	—	Modelo/tamaño del wing utilizado
Tabla	str	—	Modelo de tabla utilizado
Foil	str	—	Modelo de foil utilizado
Sensación	int	1-10	Sensación subjetiva de la sesión

2.2. Datos ingresados por el usuario en tiempo real

- Nueva sesión: todos los campos de la tabla anterior, ingresados por consola. El viento y dirección se obtienen automáticamente via API si hay conexión.
- Condiciones del día para recomendación: ubicación, velocidad del viento y dirección.

3. OUTPUTS

3.1. Recomendación de wing

- Wing recomendado basado en las sesiones más similares.
- Sensación promedio esperada para esas condiciones.
- Cantidad de sesiones similares encontradas.
- Aviso si hay menos de 3 sesiones similares o ninguna.

3.2. Métricas generales

- Cantidad total de sesiones.
- Viento promedio, sensación promedio y duración promedio.
- Cantidad de ubicaciones distintas y días navegados.

3.3. Resumen rápido

- Total de sesiones y sensación promedio del historial.

3.4. Gráficos

- Uso de wings: cuántas veces se usó cada wing (barras).
- Sensación promedio por wing (barras).

3.5. Mensajes de error

- Viento inválido: se avisa al usuario y se vuelve al menú.
- Opción inválida: se avisa al usuario y se vuelve al menú.
- Dataset no encontrado: el programa termina con exit().
- API inaccesible: se activa ingreso manual de viento y dirección.

4. PROCESOS PRINCIPALES

1. Carga del dataset

Al iniciar, el programa carga el Excel con pandas y elimina filas con valores nulos. Si no se puede cargar, termina.

2. Registro de nueva sesión

El usuario ingresa los datos por consola. El programa consulta la API de OpenWeatherMap para obtener viento y dirección automáticamente. Si la API falla, el usuario los ingresa manualmente. Se validan los datos, se agregar la fila al DataFrame y se sobrescribe el Excel.

3. Búsqueda de sesiones similares

Filtra el historial por misma ubicación, misma dirección de viento y velocidad similar (± 3 nudos).

4. Cálculo de la recomendación

Agrupar las sesiones similares por wing, calcula sensación promedio de cada uno y recomienda el de mayor promedio.

5. Cálculo de métricas

Procesa el historial completo para calcular totales promedios y frecuencias.

6. Generación de gráficos

Genera los gráficos con matplotlib desde visualizaciones.py.

5. ESTRUCTURAS DE DATOS

5.1. Dataset en memoria

El historial se carga como un DataFrame de pandas. Cada fila es una sesión y cada columna corresponde a un campo del registro. Se usa pandas por la facilidad de filtrado, agrupación y cálculo de promedios.

5.2. Resultado de la recomendación

La función recomendar_wing imprime directamente en consola:

```
===== RECOMENDACIÓN =====  
Wing recomendado: 5m  
Sensación esperada: 8.3  
Basado en 3 sesiones similares
```

6. MÓDULOS Y FUNCIONES

Módulo	Responsabilidad
-----	-----
carga_datos.py	→ Cargar Excel, limpiar datos y registrar sesiones nuevas
validaciones.py	→ Validar datos ingresados (retorna True/False)
recomendaciones.py	→ Buscar sesiones similares y recomendar wing
analisis_datos.py	→ Calcular y mostrar métricas y resumen
visualizaciones.py	→ Generar gráficos de progreso
api_clima.py	→ Consultar viento y dirección via OpenWeatherMap y convertir a punto cardinal
main.py	→ Orquestrar el flujo y manejar el menú
app.py	→ Módulo auxiliar del proyecto

carga_datos.py

- cargar_dataset()
Carga el Excel desde data/dataset_sesiones.xlsx y retorna un DataFrame. Retorna None si falla.
- verificar_columnas(df)
Verifica que el DataFrame tenga todas las columnas esperadas.
Retorna True/False.
- limpiar_datos(df)
Elimina filas con valores nulos, fuerza tipos numéricos en las columnas Vel. Viento (kn), Sensación y Duración (min).
Retorna el DataFrame limpio.

- registrar_sesion(df_actual, datos_nuevos)
Agrega una nueva sesión al DataFrame y sobrescribe el Excel.
Retorna el DataFrame actualizado.

validaciones.py

- validar_viento(viento)
Retorna True si el viento es un número positivo, False si no.
- validar_sensacion(sensacion)
Retorna True si la sensación es un entero entre 1 y 10.
- validar_duracion(duracion)
Retorna True si la duración es un número positivo.
- validar_opcion(opcion)
Retorna True si la opción es válida en el menú (0-3).

recomendaciones.py

- buscar_sesiones_similares(df, ubicacion, viento, direccion)
Filtrar el DataFrame por ubicación, dirección y viento (± 3 nudos). Retorna un DataFrame con las sesiones similares.
- recomendar_wing(df, ubicacion, viento, direccion)
Llama a buscar_sesiones_similares, agrupa por wing, calcula sensación promedio y muestra la recomendación por consola.

analisis_datos.py

- mostrar_metricas(df)
Calcula y muestra: cantidad de sesiones, viento promedio, sensación promedio, ubicaciones distintas, días navegados y duración promedio.
- mostrar_resumen(df)
Muestra un resumen rápido: total de sesiones y sensación promedio del historial.

visualizaciones.py

- grafico_wings(df)
Gráfico de barras con la cantidad de veces que se usó cada wing.
- grafico_sensacion_promedio(df)
Gráfico de barras con sensación promedio por wing.

api_clima.py

- validar_ubicacion(ubicacion)
Valida que la ubicación sea un texto no vacío.

Lanza ValueError si no es válida.

- consultar_clima(ubicacion, api_key)
Consulta OpenWeatherMap y retorna el JSON con los datos meteorológicos. Lanza errores si falla la conexión o la ubicación no existe.
- grados_a_cardinal(grados)
Convierte grados (0-360) a punto cardinal en español (Norte, Sur, Este, Oeste, Noreste, Sudeste, Noroeste, Sudoeste).
- procesar_datos_viento(datos_json)
Extrae velocidad (m/s a nudos) y dirección (grados a cardinal) del JSON. Retorna (viento_nudos, direccion).

main.py

- Carga el dataset al iniciar. Si falla, termina con exit().
- Limpia el dataset con limpiar_datos.
- Menú con opciones:
 - 1 - Ver métricas
 - 2 - Recomendar wing
 - 3 - Registrar sesión nueva (con API o manual)
 - 0 - Salir
- Si la opción es inválida, avisa y vuelve al menú.

7. ERRORES POSIBLES

Error	Origen	Cómo se maneja

Viento no numérico/negativo	→ Input incorrecto	: validar_viento False; main.py avisa
Sensación fuera de rango	→ Input incorrecto	: validar_sensacion False; main.py avisa
Duración negativa o cero	→ Input incorrecto	: validar_duracion False; main.py avisa
Opción de menú inválida	→ Input incorrecto	: main.py avisa y vuelve
Archivo Excel no encontrado	→ Ruta incorrecta	: cargar_dataset None; main.py hace exit()
API inaccesible	→ Sin conexión	: except activa ingreso manual
Ubicación no encontrada	→ API error 404	: NameError en api_clima; capturado en main.py
Menos de 3 sesiones	→ Poco historial	: recomendar_wing avisa
Sin sesiones similares	→ Sin coincidencias	: recomendar_wing informa

8. CRITERIOS DE CALIDAD

- Separación de responsabilidades: cada módulo tiene una única función. main.py no contiene lógica de análisis ni cálculo.
- Sin prints en funciones de cálculo: analisis_datos.py, recomendaciones.py y visualizaciones.py muestran resultados directamente al usuario por diseño.

- Manejo de errores: validaciones retornan True/False; main.py interpreta y actúa. API con try/except en main.py.
- Docstrings en todas las funciones.
- Código legible: nombres descriptivos, coherentes con el dominio del proyecto.
- Dataset con más de 1000 registros en data/dataset_sesiones.xlsx.
- El programa no se rompe con entradas inválidas del usuario.

Estructura de carpetas:

```
wingfoil/  
├── data/  
│   └── dataset_sesiones.xlsx  
├── diagramas/  
│   ├── estructura_general.drawio.png  
│   ├── main.drawio.png  
│   ├── recomendar_wing.drawio.png  
│   └── registrar_sesion.drawio.png  
├── src/  
│   ├── analisis_datos.py  
│   ├── api_clima.py  
│   ├── carga_datos.py  
│   ├── recomendaciones.py  
│   ├── validaciones.py  
│   └── visualizaciones.py  
├── app.py  
├── main.py  
├── requirements.txt  
└── README.md
```
